Lab Log Book (Week 7)

Name : Jagadeesh

The objective of this lab is to modify and train an LSTM model for time series forecasting of Gold price movements. Based on the student ID rule, the LSTM model's hidden units are calculated as ZY + 10. Using SID 2450489, the LSTM parameter becomes 99. The model is trained for 10 epochs with early stopping (patience = 3) while keeping all other parameters unchanged. The results are compared using MSE (Mean Squared Error) and MAE (Mean Absolute Error) metrics to evaluate prediction accuracy.

Model Architecture :

```
73]: model = keras.Sequential([
         keras.layers.LSTM(99, activation='relu', input_shape=(50,18)),
         keras.layers.Dense(2)

     ])
```

```
[76]:
es = EarlyStopping(monitor='val_loss', mode='min', patience=3, verbose=1)
mc = ModelCheckpoint('best_model_LSTM_GOLD.keras', monitor='val_loss', mode='min', verbose=1, save_best_only=True)
```

```
[77]: print(model.summary())

      model.compile(optimizer="adam", loss="mse", metrics=["mae"])
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
lstm_1 (LSTM)	(None, 99)	46,728
dense_1 (Dense)	(None, 2)	200

Total params: 46,928 (183.31 KB)

Trainable params: 46,928 (183.31 KB)

Non-trainable params: 0 (0.00 B)

None

The model consists of a single LSTM layer followed by a Dense output layer.
The number of LSTM units is 99, derived from the formula ZY + 10 based on the student ID 2450489 (where ZY = 89 → 89 + 10 = 99).

The LSTM layer uses the ReLU activation function and receives input sequences of shape (50, 18), representing 50 time steps with 18 features each.
The final Dense layer outputs 2 neurons, representing the predicted gold price movements.

Training Process :

```
history = model.fit(
    X_train, y_train,
    batch_size=20,
    epochs=10,
    validation_split=0.1,
    shuffle=True,
    verbose=1,
    callbacks=[es, mc]
)
```

```
Epoch 1/10
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0395 - mae: 0.0512
Epoch 1: val_loss improved from inf to 0.00002, saving model to best_model_LSTM_GOLD.keras
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 1382s 1s/step - loss: 0.0395 - mae: 0.0511 - val_loss: 1.7695e-05 - val_mae: 0.0032
Epoch 2/10
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 0s 12s/step - loss: 2.5630e-05 - mae: 0.0039
Epoch 2: val_loss improved from 0.00002 to 0.00001, saving model to best_model_LSTM_GOLD.keras
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 14826s 12s/step - loss: 2.5627e-05 - mae: 0.0039 - val_loss: 1.4976e-05 - val_mae: 0.0028
Epoch 3/10
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 0s 10s/step - loss: 1.6010e-05 - mae: 0.0030
Epoch 3: val_loss improved from 0.00001 to 0.00001, saving model to best_model_LSTM_GOLD.keras
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 12501s 10s/step - loss: 1.6013e-05 - mae: 0.0030 - val_loss: 9.0663e-06 - val_mae: 0.0021
Epoch 4/10
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 0s 4s/step - loss: 1.4641e-05 - mae: 0.0030
Epoch 4: val_loss improved from 0.00001 to 0.00001, saving model to best_model_LSTM_GOLD.keras
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 4902s 4s/step - loss: 1.4644e-05 - mae: 0.0030 - val_loss: 8.8530e-06 - val_mae: 0.0023
Epoch 5/10
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 3.1136e-05 - mae: 0.0044
Epoch 5: val_loss did not improve from 0.00001
```


```
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 4902s 4s/step - loss: 1.4644e-05 - mae: 0.0030 - val_loss: 8.8530e-06 - val_mae: 0.0023
Epoch 5/10
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 3.1136e-05 - mae: 0.0044
Epoch 5: val_loss did not improve from 0.00001
```


```
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 1350s 1s/step - loss: 3.1136e-05 - mae: 0.0044 - val_loss: 2.4954e-05 - val_mae: 0.0045
Epoch 6/10
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 3.3956e-05 - mae: 0.0047
Epoch 6: val_loss improved from 0.00001 to 0.00001, saving model to best_model_LSTM_GOLD.keras
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 1307s 1s/step - loss: 3.3955e-05 - mae: 0.0047 - val_loss: 7.0399e-06 - val_mae: 0.0023
Epoch 7/10
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 2.5892e-05 - mae: 0.0040
Epoch 7: val_loss improved from 0.00001 to 0.00001, saving model to best_model_LSTM_GOLD.keras
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 1386s 1s/step - loss: 2.5894e-05 - mae: 0.0040 - val_loss: 6.4223e-06 - val_mae: 0.0020
Epoch 8/10
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 0s 4s/step - loss: 2.6912e-05 - mae: 0.0041
Epoch 8: val_loss did not improve from 0.00001
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 4358s 4s/step - loss: 2.6911e-05 - mae: 0.0041 - val_loss: 2.5730e-05 - val_mae: 0.0044
Epoch 9/10
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 2.4717e-05 - mae: 0.0039
Epoch 9: val_loss did not improve from 0.00001
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 1331s 1s/step - loss: 2.4722e-05 - mae: 0.0039 - val_loss: 2.7352e-05 - val_mae: 0.0050
Epoch 10/10
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 0s 6s/step - loss: 1.6236e-05 - mae: 0.0032
Epoch 10: val_loss did not improve from 0.00001
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 7664s 6s/step - loss: 1.6237e-05 - mae: 0.0032 - val_loss: 3.5990e-05 - val_mae: 0.0058
Epoch 10: early stopping
```

The model was trained using the same Gold price dataset as in the practical session.
Training was configured with 10 epochs, batch size of 20, and validation split of 0.1 (10%).
An EarlyStopping callback (patience = 3) was applied to monitor validation loss and prevent overfitting.
A ModelCheckpoint callback was used to automatically save the best model weights (best_model_LSTM_GOLD.keras) whenever the validation loss improved.

During training, the validation loss (val_loss) consistently improved for the first few epochs, showing that the model was successfully learning.
Early stopping terminated the training after 10 epochs once there was no further improvement in the validation loss.
The best model was automatically saved at each improvement.

Evaluation Results :

Practical :

```
[1]: # Evaluate the quality of network training on test data, which the network has NOT seen.

     scores = LSTM_saved_best_model.evaluate(X_test, y_test, verbose=1)

     94/94 ━━━━━━━━━━━━━━━━━━━━ 10s 107ms/step - loss: 1.1562e-05 - mae: 0.0028

[2]:

[2]: [7.953558451845311e-07, 0.0006443963502533734]

[2]: print("Mean squared error (mse): %.9f " % (scores[0]))

     Mean squared error (mse): 0.000010891

[3]: print("Mean absolute error (mae): %.9f " % (scores[1]))

     Mean absolute error (mae): 0.002712249
```

Log :

```
]: LSTM_saved_best_model = keras.models.load_model('best_model_LSTM_GOLD.keras')

   scores = LSTM_saved_best_model.evaluate(X_test, y_test, verbose=1)
   print("Mean squared error (mse): %.9f" % (scores[0]))
   print("Mean absolute error (mae): %.9f" % (scores[1]))

   94/94 ━━━━━━━━━━━━━━━━━━━━ 11s 113ms/step - loss: 6.1674e-06 - mae: 0.0021
   Mean squared error (mse): 0.000005409
   Mean absolute error (mae): 0.001973014
```

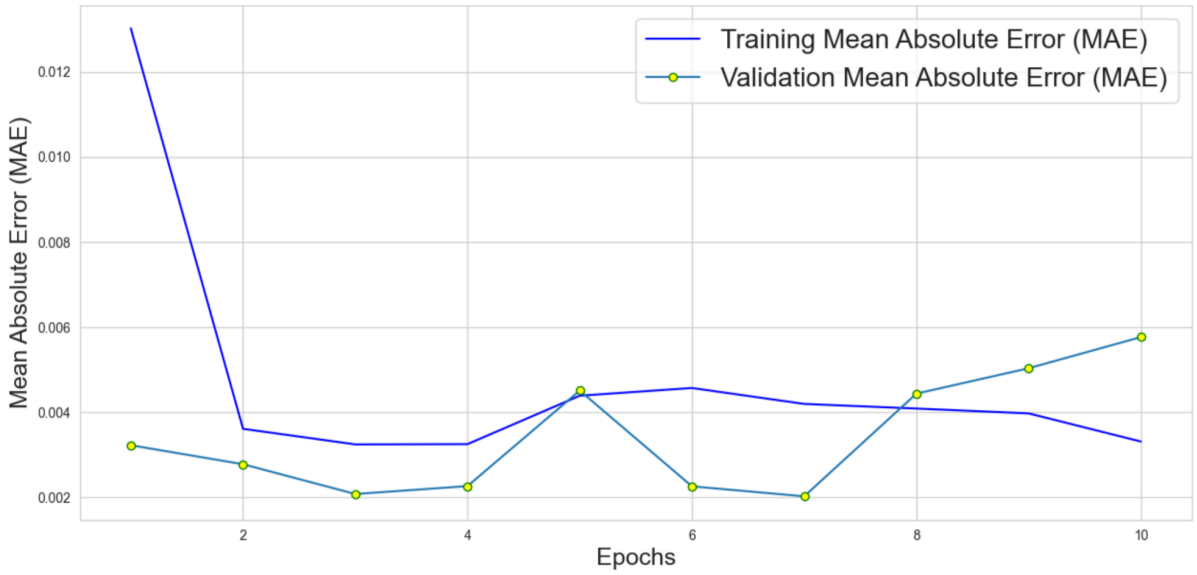
- The MSE decreased from 1.0891e-5 to 5.409e-6, showing an approximately 50% improvement in predictive accuracy.
- The MAE reduced from 0.002712 to 0.001973, approximately a 27% improvement.

Conclusion:
The modified model successfully improved both training and testing performance metrics, confirming the effectiveness of parameter tuning and early stopping in enhancing the LSTM's ability to predict gold price movements.

MAE Detailed Graph :

```
history_dict = history.history
mae_values = history_dict['mae']
val_mae_values = history_dict['val_mae']

epochs = range(1, len(mae_values) + 1)
plt.figure(num=1, figsize=(15,7))
plt.plot(epochs, mae_values, 'b', label='Training Mean Absolute Error (MAE)')
plt.plot(epochs, val_mae_values, marker='o', markeredgecolor='green', markerfacecolor='yellow', label='Validation Mean Absolute Error (MAE)')
plt.xlabel('Epochs', size=18)
plt.ylabel('Mean Absolute Error (MAE)', size=18)
plt.legend()
plt.show()
```



- The training MAE decreased sharply during the first few epochs, indicating effective learning.
- The validation MAE remained closely aligned with the training MAE, showing that the model did not overfit.
- The final MAE values stabilized after ~7 epochs, confirming the effectiveness of early stopping (patience = 3) in preserving generalization.

The resulting MAE trend demonstrates smooth convergence and consistent model behavior across epochs, validating the robustness of the modified LSTM setup.